



**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering**

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.5.1

Automatic ON-OFF Temperature Control with Hysteresis

on Embedded Systems

Checked by:

Martîniuc Alexei, *univ. asist.*
FCIM, UTM

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of this laboratory work is to design, implement, and document a modular embedded control application that regulates a measurable physical parameter by means of ON-OFF control with hysteresis. The chosen individual variant is temperature control with a DHT22 digital sensor and a relay actuator. The application runs on an Arduino Mega 2560, uses FreeRTOS tasks for acquisition, control, actuation, input, and display, and reports the relevant signals to both an LCD 1602 module and the Arduino Serial Plotter.

Unlike a purely simulated laboratory exercise, this work is prepared for a real Arduino wiring setup. A Wokwi project is still included as a reproducible pin-mapping reference, but the validation evidence is expected to come from the physical circuit assembled with a DHT22 sensor, potentiometer, keypad, LCD, relay module, external load supply, LEDs, and breadboard wiring.

1.2. Problem Definition

“Să se proiecteze și să se implementeze o aplicație modulară pentru microcontroler (MCU) care realizează controlul ON-OFF cu histereză asupra unui parametru fizic. Sistemul va acționa un actuator în funcție de valoarea măsurată a parametrului, comparată cu o valoare de referință configurabilă. Setarea valorii de referință se va face printr-un mecanism de interacțiune ales, iar valorile parametrului măsurat, ale Set Point-ului și starea actuatorului vor fi afișate pe LCD și/sau prin interfața serială. Pentru demonstrarea controlului în timp real, datele relevante (SetPoint, Value, Output) vor fi trimise către Arduino Serial Plotter.”

The implemented system follows Variant A from the laboratory task. The controlled physical parameter is the ambient temperature measured by the DHT22 sensor. The relay represents a binary heating actuator: it is energized when the measured temperature falls below the lower hysteresis threshold and de-energized when the temperature rises above the upper threshold. The reference value can be set from a potentiometer or switched to manual keypad control. The keypad also supports numeric setpoint entry and hysteresis-band adjustment.

The functional requirements are the following: acquire DHT22 temperature and humidity periodically; read the potentiometer and map it to a configurable temperature setpoint; allow keypad interaction for manual setpoint mode; compute an ON-OFF control command with hysteresis; actuate the relay safely; display temperature, setpoint, thresholds, sensor validity, and relay state on the LCD; and print ‘SetPoint’, ‘Value’, ‘Output’, ‘Low’, and ‘High’ fields in a format accepted by Arduino Serial Plotter.

The non-functional requirements are modularity, deterministic task recurrence, clear separation between drivers and application logic, reuse of existing repository libraries, preservation of previous labs, and a hardware-first validation workflow.

1.3. Technical Requirements and Validation Criteria

The laboratory guide requires not only an implementation, but also a clear connection between requirements and validation scenarios. For this system, the main validation question is whether the measured temperature, configurable setpoint, hysteresis thresholds, and relay command remain consistent across the LCD, Serial Plotter, and physical actuator.

ID	Requirement	Validation Criterion
R1	Acquire temperature and humidity from DHT22 without blocking other tasks unnecessarily.	LCD and serial output update every acquisition cycle; invalid reads set ‘Valid:0’.
R2	Provide a configurable temperature setpoint through potentiometer and keypad.	Potentiometer changes ‘Set-Point’; keypad manual entry overrides it after confirmation.
R3	Apply ON-OFF control with hysteresis around the selected setpoint.	Relay turns ON below T_{low} , turns OFF above T_{high} , and holds state inside the band.
R4	Separate acquisition, control, actuation, display, and input into modules/tasks.	Source tree contains dedicated task files and reusable library modules.
R5	Report SetPoint, measured Value, and Output for real-time analysis.	Arduino Serial Plotter receives named fields ‘SetPoint’, ‘Value’, and ‘Output’.
R6	Preserve safe behavior on sensor failure.	Invalid DHT22 read forces relay command OFF and displays sensor error status.

Table 1: Technical requirements and validation criteria

1.4. Used Technologies

ON-OFF Control with Hysteresis

ON-OFF control is a binary control method that drives an actuator either fully active or fully inactive. In a temperature-control context, a heating actuator is enabled when the measured temperature is too low and disabled when the temperature has recovered. Without hysteresis, relay contacts may chatter around the setpoint because small measurement fluctuations can repeatedly cross the same threshold.

Hysteresis solves this by defining two thresholds around the setpoint. For this implementation, the lower threshold is

$$T_{low} = SP - \frac{H}{2}$$

and the upper threshold is

$$T_{high} = SP + \frac{H}{2}$$

where SP is the selected setpoint and H is the hysteresis band. The relay turns ON when $T \leq T_{low}$, turns OFF when $T \geq T_{high}$, and preserves its previous state between the two thresholds.

DHT22 Single-Wire Sensor Interface

The DHT22 sensor provides digital temperature and humidity measurements through a timing-based single-wire protocol. In this system, the DHT22 data line is connected to digital pin D2. The software uses the Adafruit DHT library through the reusable ‘DhtSensor‘ wrapper, which stores the last valid temperature and humidity samples and marks failed reads explicitly.

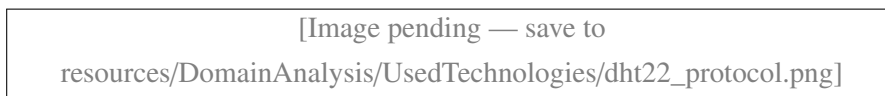


Figure 1.1 – DHT22 single-wire communication concept

I2C LCD Interface

The LCD 1602 display uses an I2C backpack, reducing the required microcontroller wiring to SDA, SCL, 5 V, and GND. On the Arduino Mega 2560, SDA and SCL are routed to the dedicated I2C pins. The report and implementation use the same LCD address, 0x27, and the reusable ‘LcdDisplay‘ wrapper already present in the repository.



Figure 1.2 – I2C protocol used by the LCD module

FreeRTOS Task Scheduling

FreeRTOS is used to structure the application into predictable tasks. Acquisition runs every 2000 ms to respect the DHT22 sampling interval, keypad input is scanned every 50 ms, display reporting runs every 500 ms, and control and actuation are event-driven through semaphores. A mutex protects shared state between tasks.

1.5. Hardware Components

Arduino Mega 2560

The Arduino Mega 2560 is based on the ATmega2560 MCU. It provides enough GPIO pins for the keypad matrix, relay input, DHT22 data line, potentiometer input, and I2C LCD. The board runs at 16 MHz and provides the 5 V logic level used by the selected peripherals.



Figure 1.6 – 4x4 membrane matrix keypad

LCD 1602 I2C Display

The LCD provides immediate feedback without a computer connection. It alternates between current control values and threshold diagnostics.



Figure 1.7 – LCD 1602 with I2C backpack

Relay Module and External Load Supply

The relay module is the binary actuator. Its logic side is controlled from D12, and its contact side switches an external circuit represented by LEDs and a separate supply in the physical wiring. A relay was chosen because ON-OFF control is naturally compatible with binary loads such as heaters, lamps, fans, or pumps.

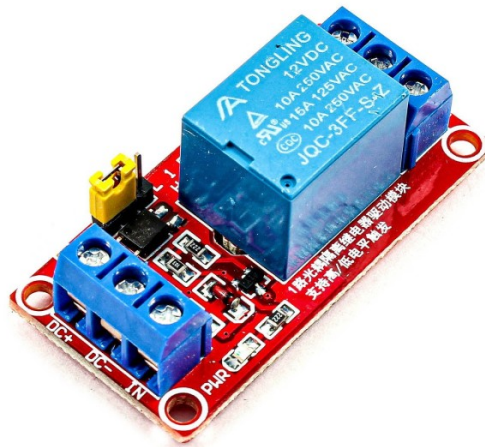


Photo by ElectroPeak

Figure 1.8 – Relay module used as binary actuator

[Photo pending — save to
resources/DomainAnalysis/Hardware-
Components/battery_9v.png]

Figure 1.9 – External 9 V battery used by the relay-switched load branch

LED Indicators, Resistors, and Breadboard

LEDs with current-limiting resistors represent the switched load states on the breadboard. The breadboard provides the temporary interconnection structure used for physical validation.

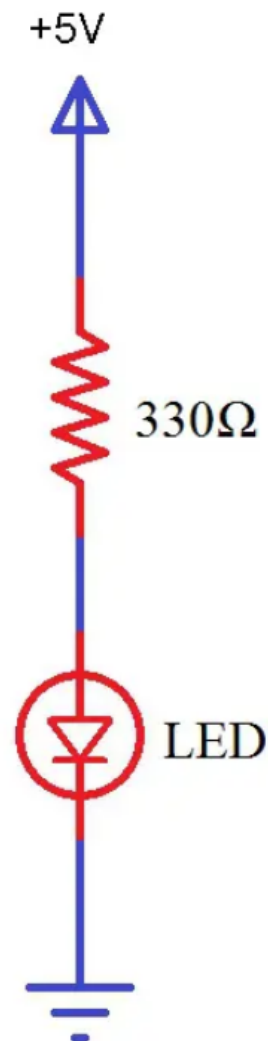


Figure 1.10 – LED used as relay-switched visual load

1.6. Software Components

PlatformIO

PlatformIO is the build and dependency-management environment used by the repository. Lab 5.1 is added as its own environment, 'lab5_1', so it can be built without overwriting earlier labs.

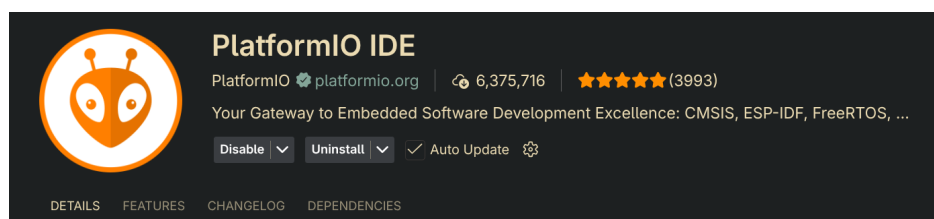


Figure 1.11 – PlatformIO IDE in VS Code

Wokwi Wiring Reference

Wokwi is included to preserve a machine-readable circuit representation and pin map. Because this laboratory is intended to be executed on a real Arduino system, the Wokwi circuit is treated as a documentation and wiring reference rather than the primary validation environment.

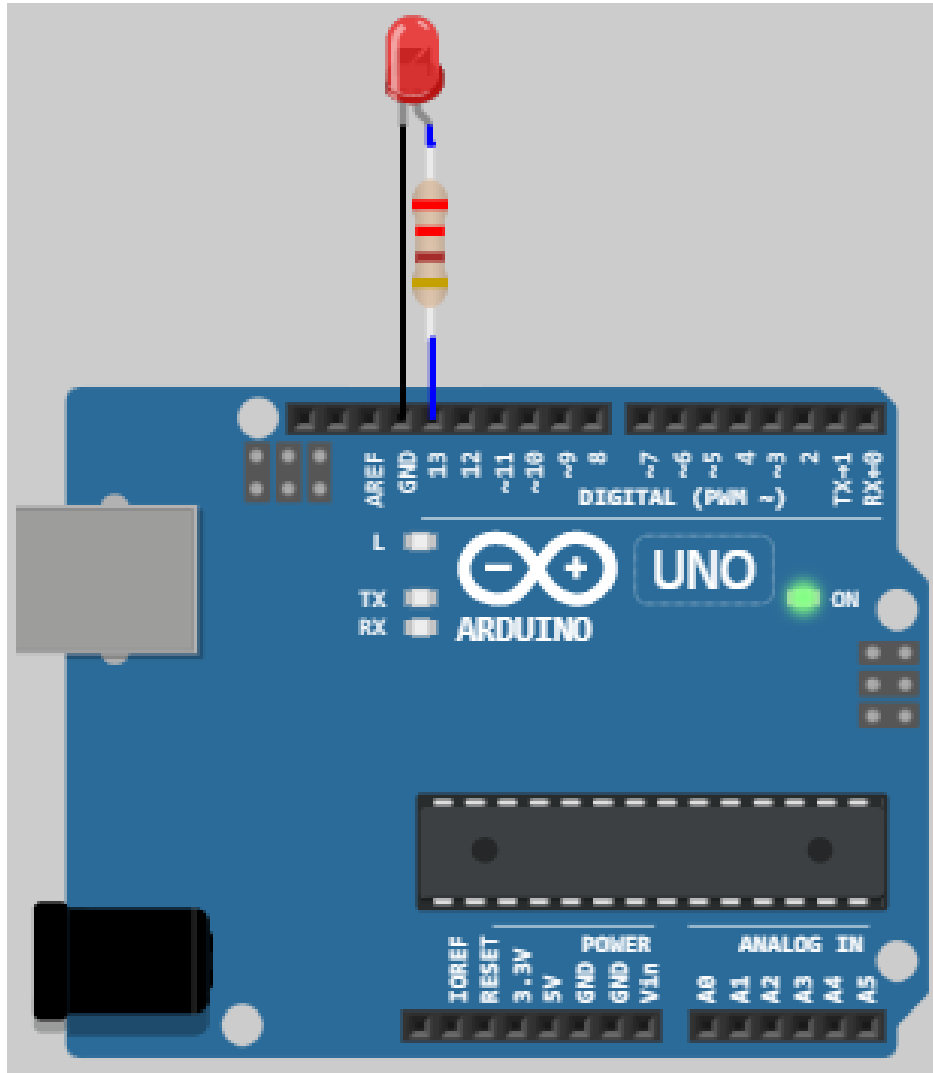


Figure 1.12 – Wokwi environment used as wiring reference

1.7. System Architecture and Justification

The architecture separates the control system into application orchestration, service-level control logic, reusable ECAL wrappers, and low-level Arduino libraries. This separation keeps the lab-specific code focused on task scheduling and state flow, while drivers such as LCD, keypad, relay, DHT22, and analog setpoint reading remain reusable.

The DHT22 is intentionally sampled every 2000 ms, because the sensor is not designed for high-rate acquisition. The control and relay actuation tasks are event-driven by semaphores, so the actuator reacts immediately after a new valid measurement is processed. The display task runs faster than the acquisition task in order to keep keypad entry and state changes visible on the LCD.

1.8. Case Study - Relevant Applicability of the Solution

ON-OFF control with hysteresis is common in thermostats, incubators, greenhouse ventilation, refrigeration systems, and pump control. In each case, the actuator usually has mechanical or thermal inertia, making frequent switching undesirable. Hysteresis protects relay contacts, reduces audible and electrical noise, and produces stable behavior with minimal computational cost. The same design pattern can be adapted from temperature regulation to humidity control, liquid-level control, fan triggering, and industrial alarm thresholds.

2. Design

2.1. System Architecture Diagrams

Component-Level Architecture

The component-level architecture is centered on the Arduino Mega 2560. The DHT22 and potentiometer provide measured and reference inputs, the keypad provides manual interaction, the LCD and Serial Plotter expose runtime state, and the relay module applies the ON-OFF actuator output.

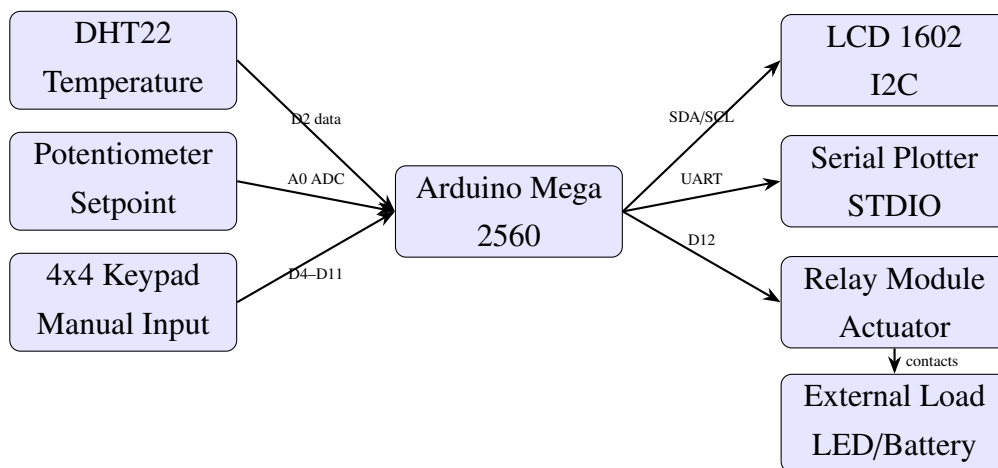


Figure 2.1 – System structural diagram of Lab 5.1

Layered System Architecture

The lab follows the repository’s layered style. The application layer creates tasks and defines behavior. The service layer owns shared state and the hysteresis algorithm. ECAL wrappers expose clean interfaces for DHT22, setpoint input, LCD, keypad, relay, and STDIO. MCAL/library code performs the actual Arduino pin and protocol operations.

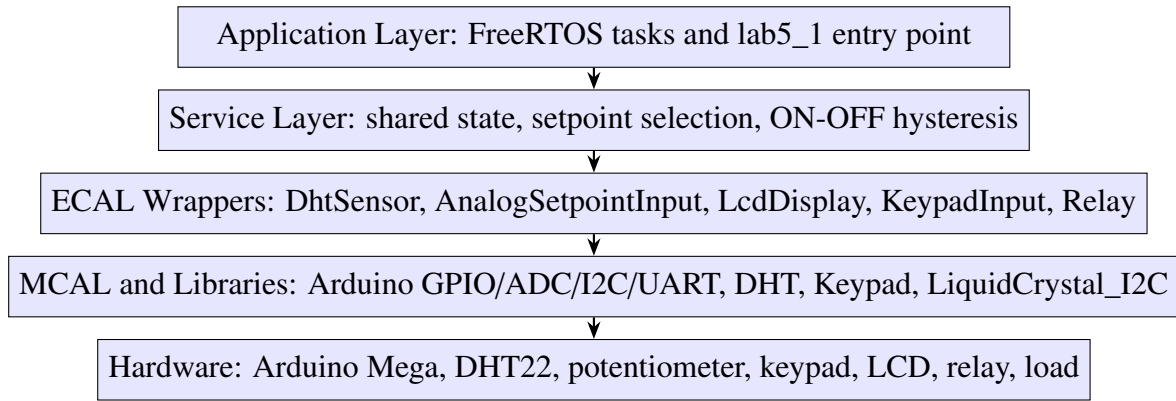


Figure 2.2 – Layered architecture of the ON-OFF control application

2.2. Block Diagrams

Runtime Data Flow

The runtime data flow is split around a mutex-protected shared state object. The input and acquisition tasks are the only tasks that produce user and sensor inputs. The control task consumes the latest sample and setpoint, then writes the desired relay command. The actuation task is the only task that touches the relay driver, and the display task periodically snapshots the state for LCD and Serial Plotter output. This makes the timing model easy to reason about because hardware writes are not scattered across unrelated modules.

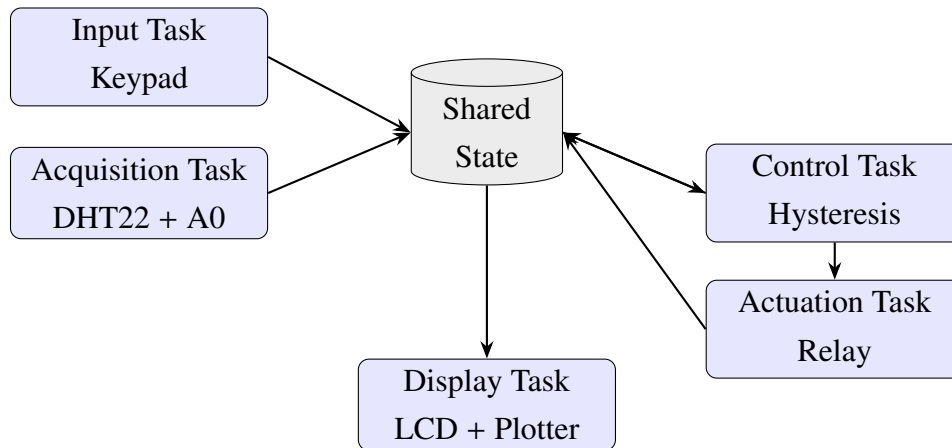


Figure 2.3 – FreeRTOS task data flow

Task Synchronization Sequence

The acquisition, control, and actuation tasks are connected through binary semaphores, while the shared state is protected by a mutex. The acquisition task gives a new-sample semaphore after writing a new DHT22 and potentiometer sample. The control task blocks on that semaphore, updates the hysteresis output, and gives an actuator semaphore. The relay task blocks on the actuator semaphore and is therefore woken only when a fresh control command is ready. The display task does not block the control path; it simply locks the mutex briefly every 500 ms to copy a snapshot.

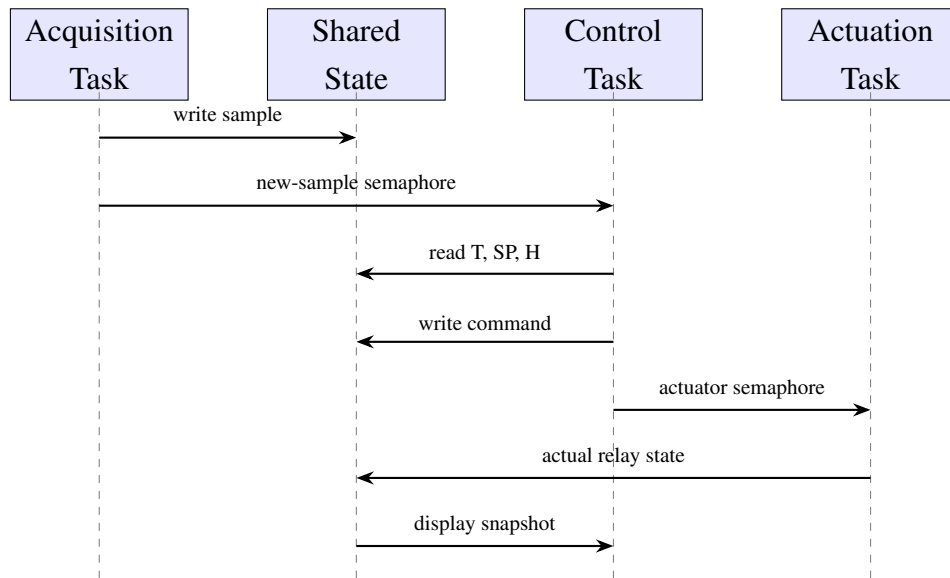


Figure 2.4 – FreeRTOS synchronization sequence for acquisition, control, and actuation

Control Algorithm Flowchart

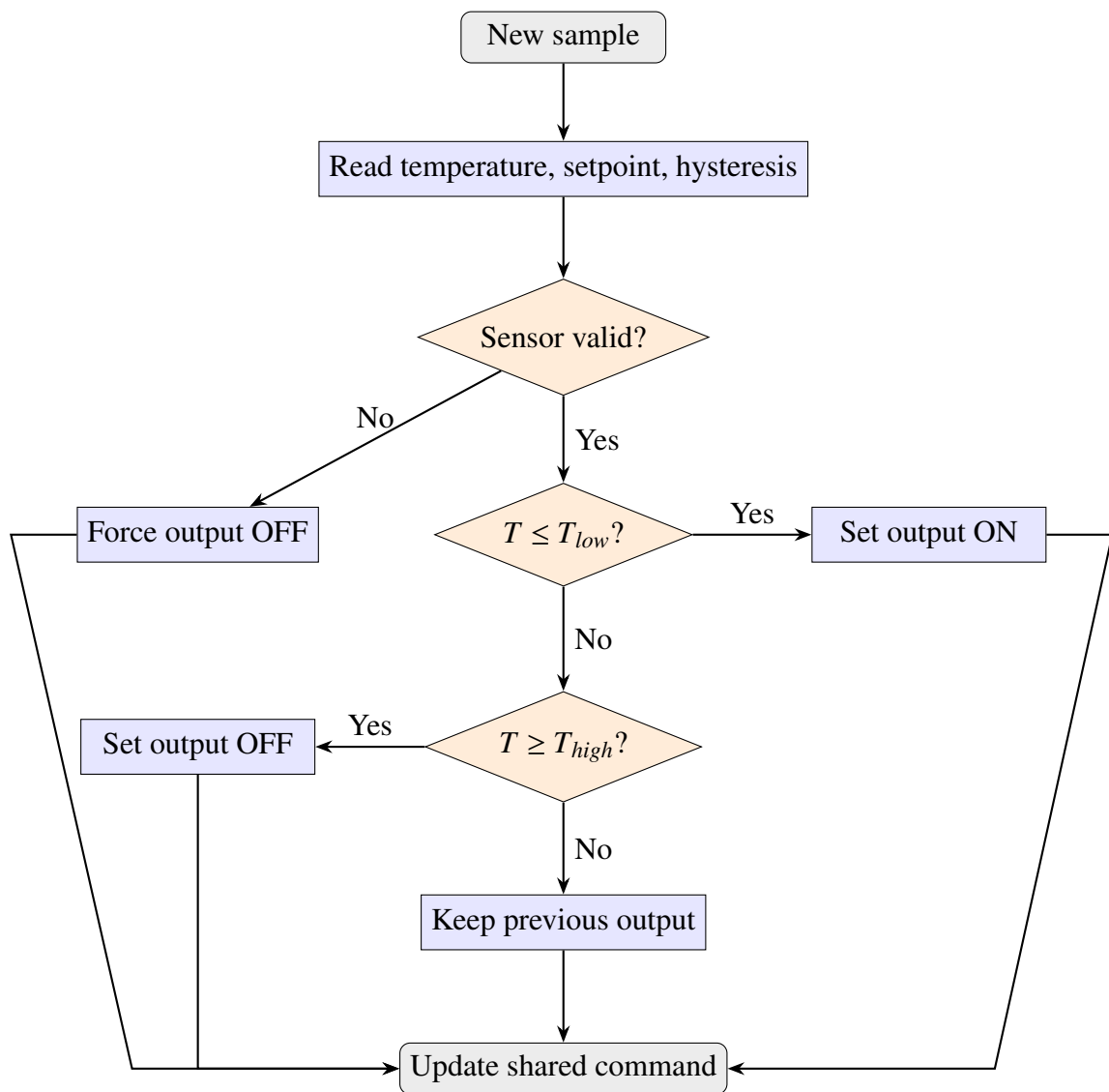


Figure 2.5 – Flowchart of the ON-OFF control algorithm with hysteresis

Controller State Diagram

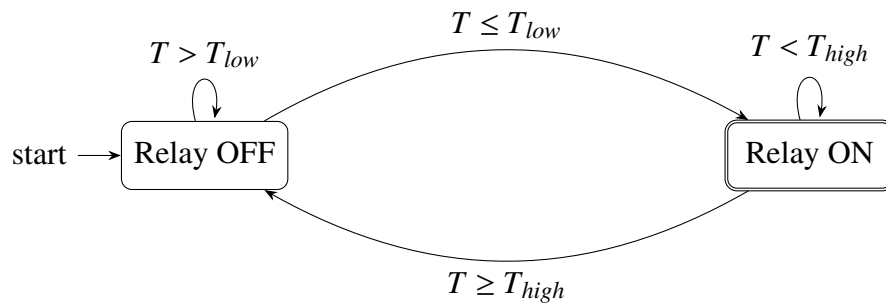


Figure 2.6 – Hysteresis controller state diagram

2.3. Electrical Schematics

Circuit Schematic

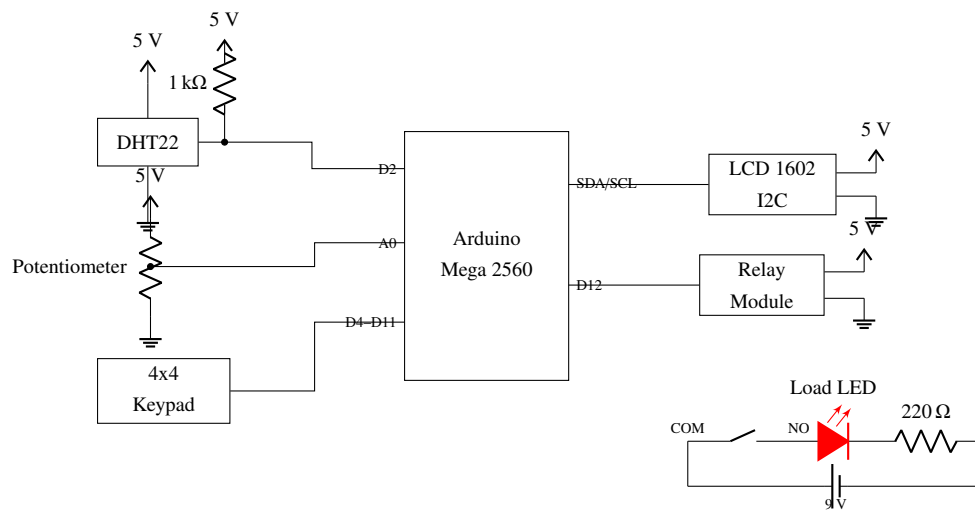


Figure 2.7 – Electrical schematic of the temperature control circuit

Component Specification

- Arduino Mega 2560: ATmega2560, 5 V logic, 16 MHz.
- DHT22: digital temperature and humidity sensor connected to D2.
- Potentiometer: analog setpoint source connected to A0.
- Keypad: 4x4 matrix, rows on D11–D8 and columns on D7–D4.
- LCD 1602 I2C: address 0x27, connected through SDA/SCL.
- Relay module: logic input on D12, active-high in the provided wiring reference.
- External load branch: 9 V battery, relay contacts, LEDs, and 220 Ω resistors.

Circuit Connections

The DHT22 VCC and GND pins are connected to the Arduino 5 V and GND rails, and its data line is wired to D2. The potentiometer is wired as a voltage divider from 5 V to GND with its wiper on A0. The keypad row pins are D11, D10, D9, and D8, while the column pins are D7, D6, D5, and D4. The LCD I2C backpack uses the Mega SDA and SCL pins. The relay module logic side is powered from 5 V and controlled by D12. The relay contacts switch the external LED load branch, which is powered separately.

Hardware Configuration

Listing 2.1 – Essential Wokwi wiring reference for Lab 5.1

```
{
  "board": "wokwi-arduino-mega",
  "sensor": { "type": "wokwi-dht22", "data": "D2" },
  "potentiometer": { "signal": "A0", "vcc": "5V", "gnd": "GND" },
  "relay": { "in": "D12", "vcc": "5V", "gnd": "GND" },
  "lcd1602": { "pins": "i2c", "sda": "SDA", "scl": "SCL" },
  "keypad": {
    "rows": ["D11", "D10", "D9", "D8"],
    "cols": ["D7", "D6", "D5", "D4"]
  }
}
```

[Screenshot pending — save the wiring-reference circuit to
resources/Design/ElectricalSchematics/wokwi_circuit.png]

Figure 2.8 – Wokwi wiring reference for the Lab 5.1 circuit

2.4. Project Structure

Listing 2.2 – Lab 5.1 project structure

```
labs/
|-- platformio.ini
|-- src/main.cpp
|-- lab/lab5_1/
|   |-- lab5_1_main.h/.cpp
|   |-- lab5_1_config.h/.cpp
|   |-- shared_state.h/.cpp
|   |-- task_input.h/.cpp
|   |-- task_acquisition.h/.cpp
|   |-- task_control.h/.cpp
|   |-- task_actuation.h/.cpp
|   '-- task_display.h/.cpp
|-- lib/
|   |-- DhtSensor/
|   |-- AnalogSetpointInput/
|   |-- OnOffHysteresisController/
|   |-- LcdDisplay/
|   |-- KeypadInput/
|   |-- Relay/
|   '-- StdioSerial/
'-- wokwi/lab5.1/
    |-- diagram.json
    '-- wokwi.toml
```

2.5. Modular Implementation

Implementation Overview

The implementation follows the same separation that was modeled in the architecture diagrams. Lab-specific behavior is located in ‘labs/lab/lab5_1’, while reusable hardware and control abstractions are located under ‘labs/lib’. The application does not call ‘digitalWrite’, ‘analogRead’, DHT protocol methods, or LCD library methods directly from the control algorithm. Instead, the low-level operations are hidden behind wrappers such as ‘DhtSensor’, ‘AnalogSetpointInput’, ‘Relay’, ‘KeypadInput’, and ‘LcdDisplay’. This keeps the control task focused on the question “what should the actuator do?” and leaves the driver modules responsible for the question “how is the hardware accessed?”

The task design also follows the extra automatic-control requirements from the laboratory guide. Acquisition, control, actuation, display, and input are separated into dedicated modules. The shared variables are protected by a FreeRTOS mutex, and the control path is coordinated through binary semaphores so a new control decision is produced only after a fresh acquisition cycle. The keypad module uses the Keypad library debounce mechanism, while the control algorithm itself uses hysteresis to suppress relay oscillations around the setpoint.

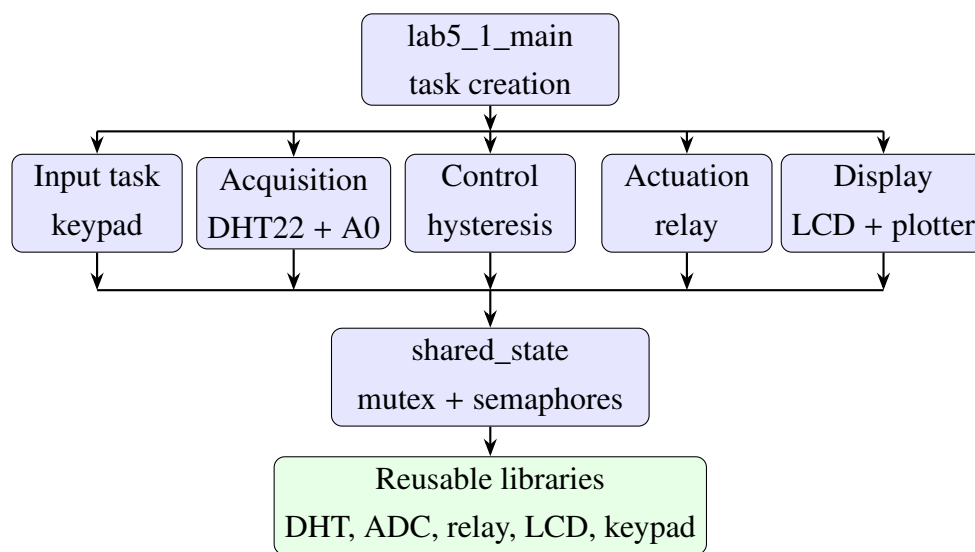


Figure 2.9 – Module responsibility map for the Lab 5.1 implementation

Application Layer: Lab Entry Point

The lab entry point is intentionally small. It initializes the STDIO serial bridge at 9600 baud, prints a startup banner that documents the keypad commands and pin mapping, initializes the shared state, and creates the five tasks. After setup completes, ‘lab5_1Loop()’ remains empty because the Arduino FreeRTOS integration transfers execution to the scheduler. This preserves the repository rule that the lab-specific entry point orchestrates the application but does not contain low-level driver logic.

Listing 2.3 – Lab 5.1 setup entry point


```

void lab5_1Setup() {
    stdioSerialInit(9600);
    lab5StateInit();
    xTaskCreate(vTaskLab5Input, "Input", TASK_INPUT_STACK,
                NULL, TASK_INPUT_PRIORITY, NULL);
    xTaskCreate(vTaskLab5Acquisition, "Acquire", TASK_ACQUISITION_STACK,
                NULL, TASK_ACQUISITION_PRIORITY, NULL);
    xTaskCreate(vTaskLab5Control, "Control", TASK_CONTROL_STACK,
                NULL, TASK_CONTROL_PRIORITY, NULL);
    xTaskCreate(vTaskLab5Actuation, "Actuate", TASK_ACTUATION_STACK,
                NULL, TASK_ACTUATION_PRIORITY, NULL);
    xTaskCreate(vTaskLab5Display, "Display", TASK_DISPLAY_STACK,
                NULL, TASK_DISPLAY_PRIORITY, NULL);
}

```

Shared State and Synchronization

The shared state module is the communication contract between tasks. It stores the most recent measured temperature, measured humidity, potentiometer raw value, potentiometer setpoint, manual setpoint, active setpoint source, hysteresis band, computed thresholds, desired relay command, actual relay state, and diagnostic counters. A mutex, ‘xLab5StateMutex’, protects this structure so that multi-field updates are seen consistently by the display and control tasks.

Two semaphores express the event order of the control chain. The new-sample semaphore is given by the acquisition task after it writes a sample. The control task waits on this semaphore, so the hysteresis calculation cannot run on stale data in a tight loop. After calculating the new command, the control task gives the actuator semaphore, which wakes the actuation task. This separation is useful on real hardware because the relay driver remains centralized in one task, while the control task remains pure decision logic.

Listing 2.4 – Shared state fields used by the control path

```

struct Lab5ControlState {
    float measuredTempC;
    float activeSetpointC;
    float hysteresisBandC;
    float lowerThresholdC;
    float upperThresholdC;
    bool sensorValid;
    bool controlCommandOn;
    bool actuatorOn;
};

```

Service Layer: Hysteresis Controller

The ‘OnOffHysteresisController’ module implements the core control law. It stores the selected setpoint, the full hysteresis band, and the previous output state. The previous output state

is important: if the measured temperature is inside the hysteresis band, the controller must not compute a new command from scratch. It must preserve its last command, otherwise the relay would still chatter around the reference.

For a heating-style actuator, the lower threshold represents the point where heat is needed, and the upper threshold represents the point where the actuator can stop. In this lab, the relay is turned ON when the measured temperature is less than or equal to the lower threshold and turned OFF when the measured temperature is greater than or equal to the upper threshold. If the sensor read is invalid, the control task calls ‘forceOutput(false)’ before updating the shared command, which makes the relay fail safe.

Listing 2.5 – ON-OFF hysteresis controller interface

```
class OnOffHysteresisController {
public:
    OnOffHysteresisController(float setpoint, float hysteresisBand);
    void init();
    void setConfig(float setpoint, float hysteresisBand);
    bool update(float measuredValue);
    void forceOutput(bool outputOn);
    bool isOutputOn() const;
    float getLowerThreshold() const;
    float getUpperThreshold() const;
};
```

Listing 2.6 – Hysteresis state update

```
bool OnOffHysteresisController::update(float measuredValue) {
    if (_state == HYSTERESIS_OUTPUT_OFF &&
        measuredValue <= getLowerThreshold()) {
        _state = HYSTERESIS_OUTPUT_ON;
    } else if (_state == HYSTERESIS_OUTPUT_ON &&
        measuredValue >= getUpperThreshold()) {
        _state = HYSTERESIS_OUTPUT_OFF;
    }
    return isOutputOn();
}
```

ECAL Layer: Acquisition Wrappers

The ‘DhtSensor’ wrapper owns DHT22 protocol access and validity checking. It calls the Adafruit DHT library, rejects ‘NaN’ readings, and exposes the last valid temperature and humidity through simple getters. This means the acquisition task does not need to know the details of the DHT timing protocol or library error values.

The ‘AnalogSetpointInput’ wrapper owns ADC acquisition and maps the potentiometer reading to the 15–35 degrees Celsius setpoint range. The conversion is linear: ADC value 0 maps to the minimum setpoint and ADC value 1023 maps to the maximum setpoint. This wrapper makes

the setpoint range easy to change from configuration without touching the task logic.

Listing 2.7 – Acquisition task sequence

```
bool sensorOk = s_dht.read();
float temperature = s_dht.getTemperatureC();
float humidity = s_dht.getHumidityPercent();
float potSetpoint = s_setpointPot.readValue();

lab5StateLock();
state->sensorValid = sensorOk;
state->measuredTempC = temperature;
state->measuredHumidityPercent = humidity;
state->potSetpointC = potSetpoint;
state->activeSetpointC =
    state->setpointSource == SETPOINT_SOURCE_POT
        ? potSetpoint
        : state->manualSetpointC;
lab5StateUnlock();
```

Input Task

The input task scans the keypad every 50 ms. The ‘KeypadInput’ wrapper configures a 20 ms debounce interval, so the task receives stable key events rather than raw electrical transitions. Key ‘A’ toggles between potentiometer and manual setpoint modes. Keys ‘B’ and ‘C’ decrement and increment the manual setpoint by 0.5 degrees Celsius. Key ‘D’ cycles the hysteresis band from 0.5 to 5.0 degrees Celsius. Digit keys start numeric setpoint entry, ‘#’ confirms the typed value, and ‘*’ cancels the edit.

The task writes only user-intent fields in shared state. It does not calculate thresholds, access the relay, or write to the LCD. That keeps interaction policy independent from control and display policy.

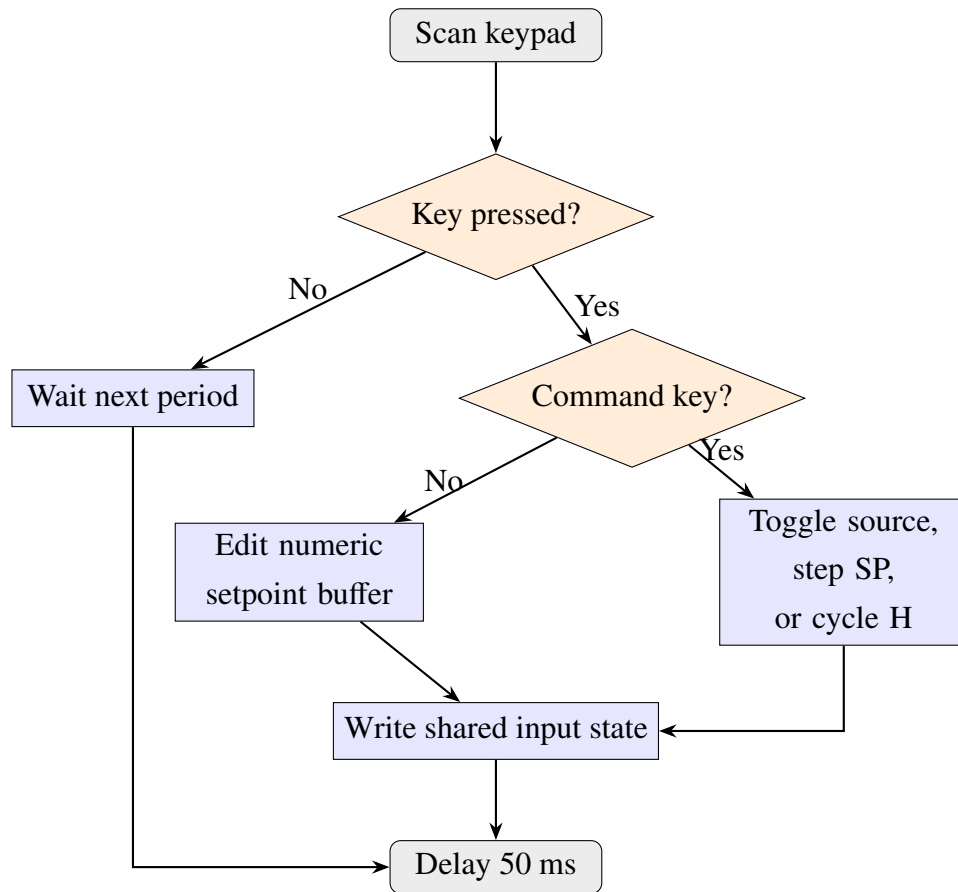


Figure 2.10 – Flowchart of keypad input processing

Acquisition Task

The acquisition task runs every 2000 ms, matching the practical sampling rate of the DHT22. Each cycle reads both sensor values and the potentiometer setpoint before taking the mutex. This is deliberate: slow I/O is performed outside the critical section, and the mutex is held only while copying values into shared state. If the selected setpoint source is the potentiometer, the active setpoint is updated from A0. If the selected source is manual, the active setpoint is copied from ‘manualSetpointC’ instead.

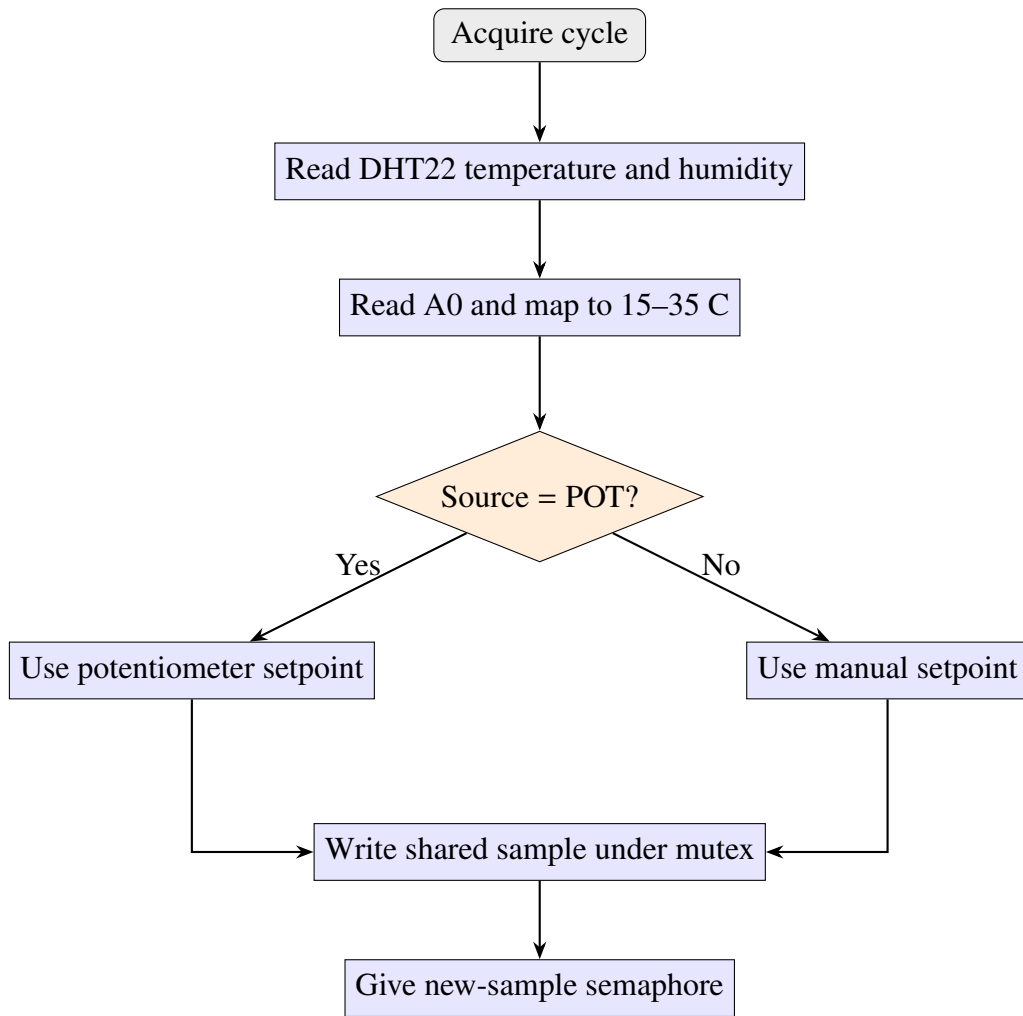


Figure 2.11 – Flowchart of the acquisition task

Control and Actuation Tasks

The control task is event-driven. It waits for a new sample, copies the current temperature, active setpoint, and hysteresis band, then updates the hysteresis controller outside the mutex. Once the command is calculated, it writes the command, controller state, lower threshold, upper threshold, and diagnostic counter back into shared state. It then gives the actuator semaphore.

The actuation task owns the ‘Relay’ instance. This is a small but important design choice because relay writes are centralized in exactly one task. The task wakes only when the control task gives the actuator semaphore, reads the desired command, applies it through the relay wrapper, and writes back the actual relay state. It also increments ‘actuatorSwitches’ whenever the physical output changes, which can later be used to evaluate relay wear or controller aggressiveness.

Listing 2.8 – Control-to-actuation handoff

```

state->controlCommandOn = commandOn;
state->lowerThresholdC = s_controller.getLowerThreshold();
state->upperThresholdC = s_controller.getUpperThreshold();
state->controlCycles++;
xSemaphoreGive(xLab5ActuatorSemaphore);

```

Display and Serial Plotter Task

The display task runs every 500 ms. It copies the shared state while holding the mutex briefly, then formats the LCD and serial output outside the critical section. This prevents slow display operations from blocking acquisition or control. The LCD alternates between two pages: a primary page with measured temperature, setpoint, relay state, and sensor status; and a diagnostics page with hysteresis band, humidity, setpoint source, and lower/upper thresholds. During numeric setpoint entry, the LCD switches to an edit page so the typed digits are visible before confirmation.

The Serial Plotter output is intentionally compact and stable. Each line uses named fields: 'SetPoint', 'Value', 'Output', 'Low', 'High', and 'Valid'. Arduino Serial Plotter can display these named channels over time, which directly satisfies the requirement to transmit SetPoint, Value, and Output for real-time analysis.

Listing 2.9 – Serial Plotter report format

```
printf("SetPoint:%s Value:%s Output:%u Low:%s High:%s Valid:%u\r\n",
      plotSetpoint,
      plotValue,
      snapshot.actuatorOn ? 1U : 0U,
      plotLow,
      plotHigh,
      snapshot.sensorValid ? 1U : 0U);
```

Timing Configuration and Justification

The selected recurrences are chosen around the hardware limitations and human interface needs. The keypad scan period is 50 ms because human button presses are slow compared with MCU execution and because the wrapper already debounces key transitions. The DHT22 acquisition period is 2000 ms to avoid invalid reads caused by excessive polling. The display period is 500 ms, fast enough to feel responsive during keypad entry but slow enough to avoid unnecessary I2C traffic. Control and actuation are event-driven rather than periodic, so the relay decision follows the newest sample without wasting CPU time between samples.

Task	Trigger	Responsibility
Input	50 ms period	Scan debounced keypad events and update setpoint/hysteresis commands.
Acquisition	2000 ms period	Read DHT22 temperature/humidity and potentiometer setpoint.
Control	New-sample semaphore	Apply ON-OFF hysteresis and publish relay command.
Actuation	Actuator semaphore	Apply relay command and report actual relay state.
Display	500 ms period	Update LCD pages and print Serial Plotter channels.

Table 2: FreeRTOS task timing and responsibility summary

3. Obtained Results

3.1. Build Process

The PlatformIO environment ‘lab5_1’ was compiled successfully for the Arduino Mega 2560 target. The successful build produced the firmware files used by the real board and by the optional Wokwi wiring reference. The build reported approximately 2056 bytes of RAM usage from 8192 bytes and 25868 bytes of flash usage from 253952 bytes.

[Screenshot pending — save successful PlatformIO build output to resources/ObtainedResults/build_results.png]

Figure 3.1 – Successful PlatformIO build output for ‘lab5_1’

3.2. Physical Hardware Setup

The laboratory is intended to be validated on real hardware. The physical setup should show the Arduino Mega, DHT22, potentiometer, keypad, LCD, relay module, breadboard, external battery, and LED load branch connected according to the pin map documented in the Design section.

[Photo pending — save the complete real Arduino setup to resources/ObtainedResults/hardware_setup.png]

Figure 3.2 – Physical Arduino setup prepared for real-hardware validation

3.3. Validation Scenarios

The validation scenarios are organized around the requirements defined in the Domain Analysis section. Because the final execution is on a physical Arduino setup, each scenario in-

cludes a placeholder for the real evidence that must be captured after flashing and running the board. The expected serial data format is also known before the physical run, because it is produced by the display task in every 500 ms cycle.

Scenario	Action	Expected Evidence
Normal operation	Power the board and wait for DHT22 acquisition.	LCD shows temperature, setpoint, relay state; Serial Plotter receives named channels.
Lower threshold crossing	Lower the setpoint above the current temperature or cool the sensor.	Output changes to 1 and relay/load branch indicates ON.
Upper threshold crossing	Warm the sensor above the upper threshold.	Output changes to 0 and relay/load branch indicates OFF.
Inside hysteresis band	Keep temperature between lower and upper thresholds.	Output remains at its previous state, proving relay chatter suppression.
Manual setpoint entry	Press digits on keypad and confirm with ‘#’.	LCD edit page appears, then Serial Plotter SetPoint changes after confirmation.

Table 3: Hardware validation scenario matrix

Normal Measurement and Display

After startup, the LCD should alternate between temperature/setpoint status and hysteresis diagnostics. The Serial Plotter line should include ‘SetPoint’, ‘Value’, ‘Output’, ‘Low’, and ‘High’.

[Screenshot/photo pending — save normal LCD/Serial operation to resources/ObtainedResults/test_normal_operation.png]

Figure 3.3 – Normal operation with temperature, setpoint, and relay state visible

Relay Turns ON Below the Lower Threshold

When the measured temperature falls below the lower threshold, the controller output must become 1 and the relay must energize. The physical load branch should indicate the switched state.

[Photo/screenshot pending — save relay ON validation to resources/ObtainedResults/test_relay_on.png]

Figure 3.4 – Relay activation after temperature drops below the lower threshold

Relay Turns OFF Above the Upper Threshold

When the measured temperature rises above the upper threshold, the controller output must return to 0 and the relay must de-energize. Values inside the hysteresis band should not change the relay state.

[Photo/screenshot pending — save relay OFF validation to
resources/ObtainedResults/test_relay_off.png]

Figure 3.5 – *Relay deactivation after temperature rises above the upper threshold*

Manual Setpoint Entry

The keypad supports switching from potentiometer setpoint mode to manual mode. Digits start numeric entry, '#' confirms the requested setpoint, and '*' cancels the entry.

[Photo/screenshot pending — save keypad setpoint test to
resources/ObtainedResults/test_manual_setpoint.png]

Figure 3.6 – *Manual setpoint entry through the 4x4 keypad*

Serial Plotter Hysteresis Trace

The Serial Plotter should show the setpoint, measured value, lower and upper thresholds, and binary output over time. The expected trace has a rectangular output signal that changes only at threshold crossings.

[Screenshot pending — save Arduino Serial Plotter trace to
resources/ObtainedResults/serial_plotter_hysteresis.png]

Figure 3.7 – *Arduino Serial Plotter trace of ON-OFF control with hysteresis*

3.4. System Observations

The implementation uses a 2000 ms acquisition period because the DHT22 should not be polled aggressively. This period is acceptable for room-temperature control, where thermal changes are slow compared with MCU execution time. The relay output is fail-safe: invalid sensor reads force the actuator OFF. The potentiometer provides fast analog setpoint adjustment, while the keypad gives repeatable manual values and hysteresis-band control.

The Wokwi project is retained as a wiring reference, but the missing battery in the simulator does not affect firmware behavior because the MCU only controls the relay coil input. The switched load branch is electrically separate from the logic branch and must be validated on the physical setup.

3.5. Verification Summary

- **Compilation:** ‘lab5_1’ builds successfully for Arduino Mega 2560.
- **Acquisition:** DHT22 temperature and humidity are sampled in a dedicated task.
- **Setpoint:** A0 potentiometer input is mapped to 15–35 degrees Celsius, with keypad override.
- **Control:** Hysteresis logic preserves relay state inside the dead band.
- **Actuation:** Relay command is applied from a separate task.
- **Display:** LCD and Serial Plotter output expose setpoint, value, thresholds, output, and validity.
- **Validation evidence:** Hardware photos and Serial Plotter screenshots must be added after running the real Arduino setup.

4. Conclusion

4.1. Achievement of Objectives

The laboratory objective was achieved by implementing a modular ON-OFF temperature-control application with hysteresis. The system acquires temperature from a DHT22 sensor, obtains a configurable setpoint from either a potentiometer or keypad input, computes lower and upper hysteresis thresholds, drives a relay actuator, and reports runtime values through LCD and Serial Plotter output.

4.2. Performance Analysis and System Limitations

The system is appropriate for slow thermal processes. The 2000 ms acquisition period protects the DHT22 from over-polling and is sufficient for ambient-temperature experiments. The hysteresis band reduces relay chatter and protects the mechanical actuator from frequent switching. A limitation is that the relay provides only binary actuation, so the controller cannot regulate power smoothly. Another limitation is that the DHT22 response time and measurement resolution limit how quickly the controller can react to sudden environmental changes.

4.3. Proposed Improvements

Future versions could add EEPROM persistence for the selected setpoint and hysteresis band, a menu page for selecting relay active level, a physical heater or fan instead of LED load indicators, and timestamped logging to a computer. For more precise thermal control, the second part of the automatic-control laboratory could replace the relay logic with a PID controller and a PWM-capable actuator.

4.4. Reflections on the Learning Experience

This laboratory demonstrates why hysteresis is essential when binary actuators are controlled from noisy or slowly changing measurements. It also reinforces the value of modular embedded design: sensor acquisition, interaction, control, actuation, and reporting can evolve independently when their responsibilities are kept separate.

4.5. Impact of Technology in Real-World Applications

The same architecture can be found in thermostats, incubators, greenhouse controllers, HVAC systems, and industrial threshold controllers. These applications often prefer simple ON-OFF control because it is robust, explainable, and inexpensive, while hysteresis provides the stability needed for mechanical actuators and real sensors.

5. Note Regarding Usage of AI

During the writing of this report and the preparation of the implementation, the author used Codex as an AI assistant to generate and consolidate LaTeX documentation, organize code structure, and check consistency between the lab requirements, pin mapping, and PlatformIO configuration. The generated material was reviewed, validated, and adjusted according to the laboratory requirements and the real-hardware setup described for this work.

6. Bibliography

- [1] Bragarenco, A., Astafi, V., *Sisteme Electronice Incorporate: Indicatii metodice pentru lucrari de laborator*, Universitatea Tehnica a Moldovei, Chisinau, 2024.
- [2] Arduino, *Arduino Mega 2560 Rev3 Documentation*, <https://docs.arduino.cc/hardware/mega-2560/>, Accessed: 2026-04-26.
- [3] Adafruit, *DHT Sensor Library*, <https://github.com/adafruit/DHT-sensor-library>, Accessed: 2026-04-26.
- [4] FreeRTOS, *FreeRTOS Kernel Documentation*, https://www.freertos.org/Documentation/RTOS_book.html, Accessed: 2026-04-26.
- [5] PlatformIO, *PlatformIO Documentation*, <https://docs.platformio.org/>, Accessed: 2026-04-26.
- [6] Wokwi, *Wokwi Arduino Mega Simulator Documentation*, <https://docs.wokwi.com/parts/wokwi-arduino-mega>, Accessed: 2026-04-26.

7. Appendix

The complete source code is organized below by architectural layer. All source files are available in the project GitHub repository.

GitHub Repository: <https://github.com/mcittkmims/es-labs>

7.1. PlatformIO Environment

Listing 7.1 – PlatformIO Lab 5.1 environment

```
[env:lab5_1]
platform = atmelavr
board = megaatmega2560
framework = arduino
monitor_speed = 9600
build_src_filter = +<*> +<../lab/lab5_1/*>
build_flags = -I lab/lab5_1 -DLAB5_1
lib_deps =
    feilipu/FreeRTOS
    marcoschwartz/LiquidCrystal_I2C@^1.1.4
    chris--a/Keypad@^3.1.1
    adafruit/DHT sensor library@^1.4.6
```

7.2. Application Layer

Listing 7.2 – Lab 5.1 main header

```
/**
 * @file lab5_1_main.h
 * @brief Lab 5.1 public entry points.
 */

#ifndef LAB5_1_MAIN_H
#define LAB5_1_MAIN_H

void lab5_1Setup();
void lab5_1Loop();

#endif // LAB5_1_MAIN_H
```

Listing 7.3 – Lab 5.1 main implementation

```
/**
 * @file lab5_1_main.cpp
 * @brief Lab 5.1 - ON-OFF temperature control with hysteresis.
 */
```

```

#include "lab5_1_main.h"
#include "lab5_1_config.h"
#include "shared_state.h"
#include "task_input.h"
#include "task_acquisition.h"
#include "task_control.h"
#include "task_actuation.h"
#include "task_display.h"

#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <stdio.h>

#include "StdioSerial.h"

void lab5_1Setup() {
    stdioSerialInit(9600);

    printf("\r\n");
    printf("=====\r\n");
    printf("  Lab 5.1 - ON-OFF Control with Hysteresis\r\n");
    printf("  Variant A: DHT22 temperature + relay actuator\r\n");
    printf("  Arduino Mega | FreeRTOS | LCD | Keypad\r\n");
    printf("=====\r\n");
    printf("KEYPAD:\r\n");
    printf("  A = toggle setpoint source POT/MANUAL\r\n");
    printf("  B/C = decrease/increase manual setpoint by 0.5 C\r\n");
    printf("  D = cycle hysteresis band 0.5..5.0 C\r\n");
    printf("  digits + # = enter integer manual setpoint\r\n");
    printf("  * = cancel numeric entry\r\n");
    printf("PINS:\r\n");
    printf("  DHT22 data: D%u\r\n", (unsigned)PIN_DHT22);
    printf("  Relay IN:   D%u\r\n", (unsigned)PIN_RELAY);
    printf("  Pot SIG:    A0\r\n");
    printf("  LCD:        SDA/SCL\r\n");
    printf("PLOTTER LINE:\r\n");
    printf("  SetPoint:<C> Value:<C> Output:<0/1> Low:<C> High:<C>\r\n");
    printf("=====\r\n\r\n");

    lab5StateInit();

    BaseType_t okInput = xTaskCreate(
        vTaskLab5Input,
        "Input",
        TASK_INPUT_STACK,
        NULL,
        TASK_INPUT_PRIORITY,
        NULL
    );
}

```

```

);

BaseType_t okAcquisition = xTaskCreate(
    vTaskLab5Acquisition,
    "Acquire",
    TASK_ACQUISITION_STACK,
    NULL,
    TASK_ACQUISITION_PRIORITY,
    NULL
);

BaseType_t okControl = xTaskCreate(
    vTaskLab5Control,
    "Control",
    TASK_CONTROL_STACK,
    NULL,
    TASK_CONTROL_PRIORITY,
    NULL
);

BaseType_t okActuation = xTaskCreate(
    vTaskLab5Actuation,
    "Actuate",
    TASK_ACTUATION_STACK,
    NULL,
    TASK_ACTUATION_PRIORITY,
    NULL
);

BaseType_t okDisplay = xTaskCreate(
    vTaskLab5Display,
    "Display",
    TASK_DISPLAY_STACK,
    NULL,
    TASK_DISPLAY_PRIORITY,
    NULL
);

if (okInput != pdPASS || okAcquisition != pdPASS ||
    okControl != pdPASS || okActuation != pdPASS ||
    okDisplay != pdPASS) {
    printf("[ERROR] Task creation failed: I=%ld A=%ld C=%ld R=%ld D=%ld\r\n",
        (long)okInput,
        (long)okAcquisition,
        (long)okControl,
        (long)okActuation,
        (long)okDisplay);
}

```

```

}

void lab5_1Loop() {
    // All application logic runs in FreeRTOS tasks.
}

```

Listing 7.4 – Lab 5.1 configuration header

```

/**
 * @file lab5_1_config.h
 * @brief Lab 5.1 configuration and pin mapping.
 */

#ifndef LAB5_1_CONFIG_H
#define LAB5_1_CONFIG_H

#include <Arduino.h>
#include <Arduino_FreeRTOS.h>

// Hardware pin mapping from the provided Arduino Mega wiring.
static const uint8_t PIN_DHT22 = 2;
static const uint8_t PIN_RELAY = 12;
static const uint8_t PIN_SETPOINT_POT = A0;

extern byte KEYPAD_ROW_PINS[4];
extern byte KEYPAD_COL_PINS[4];

static const uint8_t LCD_I2C_ADDRESS = 0x27;
static const uint8_t LCD_COLS = 16;
static const uint8_t LCD_ROWS = 2;

// Many real relay modules are active-low. The supplied Wokwi relay module
// expects active-high control; change this constant if the physical relay
// used during the lab energizes on LOW.
static const bool RELAY_ACTIVE_HIGH = true;

// Control parameters.
static const float SETPOINT_MIN_C = 15.0f;
static const float SETPOINT_MAX_C = 35.0f;
static const float SETPOINT_DEFAULT_C = 25.0f;
static const float SETPOINT_STEP_C = 0.5f;

static const float HYSTERESIS_MIN_C = 0.5f;
static const float HYSTERESIS_MAX_C = 5.0f;
static const float HYSTERESIS_DEFAULT_C = 2.0f;
static const float HYSTERESIS_STEP_C = 0.5f;

static const uint8_t SETPOINT_INPUT_MAX_DIGITS = 3;

```

```

// FreeRTOS task periods.
static const uint16_t TASK_INPUT_PERIOD_MS = 50;
static const uint16_t TASK_ACQUISITION_PERIOD_MS = 2000;
static const uint16_t TASK_DISPLAY_PERIOD_MS = 500;

// FreeRTOS task stack sizes and priorities.
static const configSTACK_DEPTH_TYPE TASK_INPUT_STACK = 256;
static const configSTACK_DEPTH_TYPE TASK_ACQUISITION_STACK = 384;
static const configSTACK_DEPTH_TYPE TASK_CONTROL_STACK = 320;
static const configSTACK_DEPTH_TYPE TASK_ACTUATION_STACK = 256;
static const configSTACK_DEPTH_TYPE TASK_DISPLAY_STACK = 640;

static const UBaseType_t TASK_INPUT_PRIORITY = 3;
static const UBaseType_t TASK_ACQUISITION_PRIORITY = 3;
static const UBaseType_t TASK_CONTROL_PRIORITY = 2;
static const UBaseType_t TASK_ACTUATION_PRIORITY = 2;
static const UBaseType_t TASK_DISPLAY_PRIORITY = 1;

#endif // LAB5_1_CONFIG_H

```

Listing 7.5 – Lab 5.1 configuration implementation

```

/**
 * @file lab5_1_config.cpp
 * @brief Lab 5.1 writable pin arrays used by the Keypad library.
 */

#include "lab5_1_config.h"

byte KEYPAD_ROW_PINS[4] = {11, 10, 9, 8};
byte KEYPAD_COL_PINS[4] = {7, 6, 5, 4};

```

7.3. Shared State and Tasks

Listing 7.6 – Shared state header

```

/**
 * @file shared_state.h
 * @brief Lab 5.1 shared state and synchronization primitives.
 */

#ifndef LAB5_1_SHARED_STATE_H
#define LAB5_1_SHARED_STATE_H

#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include "lab5_1_config.h"

```



```

#include "OnOffHysteresisController.h"

enum SetpointSource {
    SETPOINT_SOURCE_POT = 0,
    SETPOINT_SOURCE_MANUAL = 1
};

struct Lab5ControlState {
    float measuredTempC;
    float measuredHumidityPercent;
    bool sensorValid;
    uint16_t potRaw;
    float potSetpointC;
    float manualSetpointC;
    float activeSetpointC;
    SetpointSource setpointSource;

    float hysteresisBandC;
    float lowerThresholdC;
    float upperThresholdC;
    bool controlCommandOn;
    bool actuatorOn;
    HysteresisOutputState controllerState;

    bool editingSetpoint;
    char inputBuffer[SETPOINT_INPUT_MAX_DIGITS + 1];
    uint8_t inputBufferLen;

    uint32_t sampleCount;
    uint32_t controlCycles;
    uint32_t actuatorSwitches;
    TickType_t lastSampleTick;
};

extern Lab5ControlState g_lab5State;
extern SemaphoreHandle_t xLab5StateMutex;
extern SemaphoreHandle_t xLab5NewSampleSemaphore;
extern SemaphoreHandle_t xLab5ActuatorSemaphore;

void lab5StateInit();
void lab5StateLock();
void lab5StateUnlock();
Lab5ControlState* lab5StateGet();

#endif // LAB5_1_SHARED_STATE_H

```

Listing 7.7 – Shared state implementation

```
/**
```

```

* @file shared_state.cpp
* @brief Lab 5.1 shared state implementation.
*/

#include "shared_state.h"
#include "lab5_1_config.h"
#include <string.h>

Lab5ControlState g_lab5State;
SemaphoreHandle_t xLab5StateMutex = NULL;
SemaphoreHandle_t xLab5NewSampleSemaphore = NULL;
SemaphoreHandle_t xLab5ActuatorSemaphore = NULL;

void lab5StateInit() {
    memset(&g_lab5State, 0, sizeof(g_lab5State));

    g_lab5State.measuredTempC = NAN;
    g_lab5State.measuredHumidityPercent = NAN;
    g_lab5State.sensorValid = false;
    g_lab5State.manualSetpointC = SETPOINT_DEFAULT_C;
    g_lab5State.activeSetpointC = SETPOINT_DEFAULT_C;
    g_lab5State.potSetpointC = SETPOINT_DEFAULT_C;
    g_lab5State.setpointSource = SETPOINT_SOURCE_POT;
    g_lab5State.hysteresisBandC = HYSTERESIS_DEFAULT_C;
    g_lab5State.lowerThresholdC = SETPOINT_DEFAULT_C - HYSTERESIS_DEFAULT_C * 0.5f;
    g_lab5State.upperThresholdC = SETPOINT_DEFAULT_C + HYSTERESIS_DEFAULT_C * 0.5f;
    g_lab5State.controlCommandOn = false;
    g_lab5State.actuatorOn = false;
    g_lab5State.controllerState = HYSTERESIS_OUTPUT_OFF;
    g_lab5State.editingSetpoint = false;
    g_lab5State.inputBuffer[0] = '\0';
    g_lab5State.inputBufferLen = 0;

    xLab5StateMutex = xSemaphoreCreateMutex();
    xLab5NewSampleSemaphore = xSemaphoreCreateBinary();
    xLab5ActuatorSemaphore = xSemaphoreCreateBinary();
}

void lab5StateLock() {
    xSemaphoreTake(xLab5StateMutex, portMAX_DELAY);
}

void lab5StateUnlock() {
    xSemaphoreGive(xLab5StateMutex);
}

Lab5ControlState* lab5StateGet() {
    return &g_lab5State;
}

```

```
}
```

Listing 7.8 – Keypad input task

```
/**
 * @file task_input.cpp
 * @brief Lab 5.1 keypad input task implementation.
 */

#include "task_input.h"
#include "lab5_1_config.h"
#include "shared_state.h"
#include "KeypadInput.h"

#include <Arduino_FreeRTOS.h>
#include <stdlib.h>
#include <stdio.h>

static KeypadInput s_keypad(KEYPAD_ROW_PINS, KEYPAD_COL_PINS);

static float clampFloat(float value, float minValue, float maxValue) {
    if (value < minValue) return minValue;
    if (value > maxValue) return maxValue;
    return value;
}

static void enterManualMode(Lab5ControlState *state) {
    if (state->setpointSource == SETPOINT_SOURCE_POT) {
        state->>manualSetpointC = state->activeSetpointC;
    }
    state->setpointSource = SETPOINT_SOURCE_MANUAL;
    state->activeSetpointC = state->>manualSetpointC;
}

void vTaskLab5Input(void *pvParameters) {
    (void)pvParameters;

    s_keypad.init();

    TickType_t lastWake = xTaskGetTickCount();
    const TickType_t period = pdMS_TO_TICKS(TASK_INPUT_PERIOD_MS);

    for (;;) {
        char key = s_keypad.getKey();

        if (key != 0) {
            lab5StateLock();
            Lab5ControlState *state = lab5StateGet();

```

```

switch (key) {
    case 'A':
        if (state->setpointSource == SETPOINT_SOURCE_POT) {
            enterManualMode(state);
            printf("[INPUT] Setpoint source: MANUAL\r\n");
        } else {
            state->setpointSource = SETPOINT_SOURCE_POT;
            state->activeSetpointC = state->potSetpointC;
            printf("[INPUT] Setpoint source: POT\r\n");
        }
        state->editingSetpoint = false;
        state->inputBufferLen = 0;
        state->inputBuffer[0] = '\0';
        break;

    case 'B':
        enterManualMode(state);
        state->manualSetpointC = clampFloat(
            state->manualSetpointC - SETPOINT_STEP_C,
            SETPOINT_MIN_C,
            SETPOINT_MAX_C
        );
        state->activeSetpointC = state->manualSetpointC;
        printf("[INPUT] Manual setpoint decreased\r\n");
        break;

    case 'C':
        enterManualMode(state);
        state->manualSetpointC = clampFloat(
            state->manualSetpointC + SETPOINT_STEP_C,
            SETPOINT_MIN_C,
            SETPOINT_MAX_C
        );
        state->activeSetpointC = state->manualSetpointC;
        printf("[INPUT] Manual setpoint increased\r\n");
        break;

    case 'D':
        state->hysteresisBandC += HYSTERESIS_STEP_C;
        if (state->hysteresisBandC > HYSTERESIS_MAX_C) {
            state->hysteresisBandC = HYSTERESIS_MIN_C;
        }
        printf("[INPUT] Hysteresis band changed\r\n");
        break;

    case '*':
        state->editingSetpoint = false;
        state->inputBufferLen = 0;

```

```

        state->inputBuffer[0] = '\0';
        printf("[INPUT] Numeric setpoint entry cancelled\r\n");
        break;

    case '#':
        if (state->editingSetpoint && state->inputBufferLen > 0) {
            state->inputBuffer[state->inputBufferLen] = '\0';
            int requested = atoi(state->inputBuffer);
            state->manualSetpointC = clampFloat(
                (float)requested,
                SETPOINT_MIN_C,
                SETPOINT_MAX_C
            );
            state->setpointSource = SETPOINT_SOURCE_MANUAL;
            state->activeSetpointC = state->manualSetpointC;
            state->editingSetpoint = false;
            state->inputBufferLen = 0;
            state->inputBuffer[0] = '\0';
            printf("[INPUT] Manual setpoint confirmed\r\n");
        }
        break;

    default:
        if (key >= '0' && key <= '9') {
            if (!state->editingSetpoint) {
                state->editingSetpoint = true;
                state->inputBufferLen = 0;
                state->inputBuffer[0] = '\0';
            }

            if (state->inputBufferLen < SETPOINT_INPUT_MAX_DIGITS) {
                state->inputBuffer[state->inputBufferLen++] = key;
                state->inputBuffer[state->inputBufferLen] = '\0';
                printf("[INPUT] Entering setpoint: %s\r\n",
                    state->inputBuffer);
            }
        }
        break;
    }

    lab5StateUnlock();
}

vTaskDelayUntil(&lastWake, period);
}
}

```

Listing 7.9 – Acquisition task

```

/**
 * @file task_acquisition.cpp
 * @brief Lab 5.1 DHT22 and potentiometer acquisition task.
 */

#include "task_acquisition.h"
#include "lab5_1_config.h"
#include "shared_state.h"

#include "DhtSensor.h"
#include "AnalogSetpointInput.h"

#include <Arduino_FreeRTOS.h>
#include <DHT.h>

static DhtSensor s_dht(PIN_DHT22, DHT22);
static AnalogSetpointInput s_setpointPot(
    PIN_SETPOINT_POT,
    SETPOINT_MIN_C,
    SETPOINT_MAX_C
);

void vTaskLab5Acquisition(void *pvParameters) {
    (void)pvParameters;

    s_dht.init();
    s_setpointPot.init();

    TickType_t lastWake = xTaskGetTickCount();
    const TickType_t period = pdMS_TO_TICKS(TASK_ACQUISITION_PERIOD_MS);

    for (;;) {
        bool sensorOk = s_dht.read();
        float temperature = s_dht.getTemperatureC();
        float humidity = s_dht.getHumidityPercent();
        float potSetpoint = s_setpointPot.readValue();
        uint16_t potRaw = s_setpointPot.getLastRaw();

        lab5StateLock();
        Lab5ControlState *state = lab5StateGet();

        state->sensorValid = sensorOk;
        state->measuredTempC = temperature;
        state->measuredHumidityPercent = humidity;
        state->potRaw = potRaw;
        state->potSetpointC = potSetpoint;

        if (state->setpointSource == SETPOINT_SOURCE_POT) {

```

```

        state->activeSetpointC = potSetpoint;
    } else {
        state->activeSetpointC = state->manualSetpointC;
    }

    state->sampleCount++;
    state->lastSampleTick = xTaskGetTickCount();

    lab5StateUnlock();
    xSemaphoreGive(xLab5NewSampleSemaphore);

    vTaskDelayUntil(&lastWake, period);
}
}

```

Listing 7.10 – Control task

```

/**
 * @file task_control.cpp
 * @brief Lab 5.1 ON-OFF hysteresis control task implementation.
 */

#include "task_control.h"
#include "lab5_1_config.h"
#include "shared_state.h"
#include "OnOffHysteresisController.h"

#include <Arduino_FreeRTOS.h>
#include <math.h>

static OnOffHysteresisController s_controller(
    SETPOINT_DEFAULT_C,
    HYSTERESIS_DEFAULT_C
);

void vTaskLab5Control(void *pvParameters) {
    (void)pvParameters;

    s_controller.init();

    for (;;) {
        if (xSemaphoreTake(xLab5NewSampleSemaphore, portMAX_DELAY) != pdTRUE) {
            continue;
        }

        lab5StateLock();
        Lab5ControlState *state = lab5StateGet();
        float temperature = state->measuredTempC;
        float setpoint = state->activeSetpointC;

```

```

float hysteresis = state->hysteresisBandC;
bool valid = state->sensorValid && !isnan(temperature);
lab5StateUnlock();

s_controller.setConfig(setpoint, hysteresis);

bool commandOn = false;
if (valid) {
    commandOn = s_controller.update(temperature);
} else {
    // Fail safe: if the sensor reading is invalid, de-energize relay.
    s_controller.forceOutput(false);
    commandOn = false;
}

lab5StateLock();
state = lab5StateGet();
state->controlCommandOn = commandOn;
state->controllerState = s_controller.getState();
state->lowerThresholdC = s_controller.getLowerThreshold();
state->upperThresholdC = s_controller.getUpperThreshold();
state->controlCycles++;
lab5StateUnlock();

xSemaphoreGive(xLab5ActuatorSemaphore);
}
}

```

Listing 7.11 – Actuation task

```

/**
 * @file task_actuation.cpp
 * @brief Lab 5.1 relay actuation task implementation.
 */

#include "task_actuation.h"
#include "lab5_1_config.h"
#include "shared_state.h"
#include "Relay.h"

#include <Arduino_FreeRTOS.h>

static Relay s_relay(PIN_RELAY, RELAY_ACTIVE_HIGH);

void vTaskLab5Actuation(void *pvParameters) {
    (void)pvParameters;

    s_relay.init();
}

```



```

lab5StateLock();
lab5StateGet()->actuatorOn = s_relay.isOn();
lab5StateUnlock();

for (;;) {
    if (xSemaphoreTake(xLab5ActuatorSemaphore, portMAX_DELAY) != pdTRUE) {
        continue;
    }

    lab5StateLock();
    bool commandOn = lab5StateGet()->controlCommandOn;
    bool previousOn = lab5StateGet()->actuatorOn;
    lab5StateUnlock();

    s_relay.setState(commandOn);

    lab5StateLock();
    Lab5ControlState *state = lab5StateGet();
    state->actuatorOn = s_relay.isOn();
    if (state->actuatorOn != previousOn) {
        state->actuatorSwitches++;
    }
    lab5StateUnlock();
}
}

```

Listing 7.12 – Display and reporting task

```

/**
 * @file task_display.cpp
 * @brief Lab 5.1 LCD and Serial Plotter reporting task.
 */

#include "task_display.h"
#include "lab5_1_config.h"
#include "shared_state.h"
#include "LcdDisplay.h"

#include <Arduino_FreeRTOS.h>
#include <stdio.h>
#include <string.h>
#include <math.h>

static LcdDisplay s_lcd(LCD_I2C_ADDRESS, LCD_COLS, LCD_ROWS);

static void formatFloat(char *buffer, size_t size, float value,
                        signed char width, unsigned char precision,
                        const char *invalidText) {
    if (isnan(value)) {

```

```

        strncpy(buffer, invalidText, size);
        buffer[size - 1] = '\0';
    } else {
        dtostrf(value, width, precision, buffer);
    }
}

static float plotValueOrZero(float value) {
    return isnan(value) ? 0.0f : value;
}

void vTaskLab5Display(void *pvParameters) {
    (void)pvParameters;

    s_lcd.init();
    s_lcd.backlight(true);
    s_lcd.showTwoLines("Lab 5.1 ONOFF", "DHT22 + Relay");

    TickType_t lastWake = xTaskGetTickCount();
    const TickType_t period = pdMS_TO_TICKS(TASK_DISPLAY_PERIOD_MS);
    uint8_t displayCycle = 0;

    for (;;) {
        vTaskDelayUntil(&lastWake, period);
        displayCycle++;

        lab5StateLock();
        Lab5ControlState snapshot = *lab5StateGet();
        lab5StateUnlock();

        char tempStr[8];
        char humStr[8];
        char spStr[8];
        char hystStr[8];
        char lowStr[8];
        char highStr[8];

        formatFloat(tempStr, sizeof(tempStr), snapshot.measuredTempC,
                    4, 1, "--.-");
        formatFloat(humStr, sizeof(humStr), snapshot.measuredHumidityPercent,
                    4, 0, "---");
        formatFloat(spStr, sizeof(spStr), snapshot.activeSetpointC,
                    4, 1, "--.-");
        formatFloat(hystStr, sizeof(hystStr), snapshot.hysteresisBandC,
                    3, 1, "-.-");
        formatFloat(lowStr, sizeof(lowStr), snapshot.lowerThresholdC,
                    4, 1, "--.-");
        formatFloat(highStr, sizeof(highStr), snapshot.upperThresholdC,

```

```

        4, 1, "--.-");

char line0[17];
char line1[17];

if (snapshot.editingSetpoint) {
    snprintf(line0, sizeof(line0), "Set SP:%-3s C", snapshot.inputBuffer);
    snprintf(line1, sizeof(line1), "#=OK *=CLR");
} else if (((displayCycle / 4) % 2) == 0) {
    snprintf(line0, sizeof(line0), "T:%s SP:%s", tempStr, spStr);
    snprintf(line1, sizeof(line1), "Relay:%-3s %s",
        snapshot.actuatorOn ? "ON" : "OFF",
        snapshot.sensorValid ? "OK" : "SERR");
} else {
    const char *source =
        snapshot.setpointSource == SETPOINT_SOURCE_POT ? "POT" : "MAN";
    snprintf(line0, sizeof(line0), "H:%s Hum:%s%%", hystStr, humStr);
    snprintf(line1, sizeof(line1), "%s %s-%s", source, lowStr, highStr);
}

s_lcd.showTwoLines(line0, line1);

char plotSetpoint[10];
char plotValue[10];
char plotLow[10];
char plotHigh[10];
dtostrf(plotValueOrZero(snapshot.activeSetpointC), 1, 1, plotSetpoint);
dtostrf(plotValueOrZero(snapshot.measuredTempC), 1, 1, plotValue);
dtostrf(plotValueOrZero(snapshot.lowerThresholdC), 1, 1, plotLow);
dtostrf(plotValueOrZero(snapshot.upperThresholdC), 1, 1, plotHigh);

printf("SetPoint:%s Value:%s Output:%u Low:%s High:%s Valid:%u\r\n",
    plotSetpoint,
    plotValue,
    snapshot.actuatorOn ? 1U : 0U,
    plotLow,
    plotHigh,
    snapshot.sensorValid ? 1U : 0U);
}
}

```

7.4. Reusable Libraries Added for Lab 5.1

Listing 7.13 – DHT sensor wrapper header

```

/**
 * @file DhtSensor.h
 * @brief DHT temperature and humidity sensor wrapper.

```

```

*
* Provides a small reusable interface around the Arduino DHT library.
* The wrapper keeps the last valid temperature and humidity values so
* application code can distinguish sensor failures from valid samples.
*/

#ifndef DHT_SENSOR_H
#define DHT_SENSOR_H

#include <Arduino.h>
#include <DHT.h>

class DhtSensor {
public:
    /**
     * @brief Construct a DHT sensor wrapper.
     * @param dataPin GPIO pin connected to the DHT data line.
     * @param sensorType DHT11, DHT21, or DHT22 constant from DHT.h.
     */
    DhtSensor(uint8_t dataPin, uint8_t sensorType);

    /** @brief Initialize the DHT driver. */
    void init();

    /**
     * @brief Read both temperature and humidity.
     * @return true when both values are valid; false on checksum/timeout error.
     */
    bool read();

    /** @brief Get the last valid temperature in degrees Celsius. */
    float getTemperatureC() const;

    /** @brief Get the last valid relative humidity percentage. */
    float getHumidityPercent() const;

    /** @brief True if the last read produced valid values. */
    bool isValid() const;

private:
    DHT _dht;
    float _temperatureC;
    float _humidityPercent;
    bool _valid;
};

#endif // DHT_SENSOR_H

```

Listing 7.14 – DHT sensor wrapper implementation

```
/**
 * @file DhtSensor.cpp
 * @brief DHT temperature and humidity sensor wrapper implementation.
 */

#include "DhtSensor.h"
#include <math.h>

DhtSensor::DhtSensor(uint8_t dataPin, uint8_t sensorType)
    : _dht(dataPin, sensorType),
      _temperatureC(NAN),
      _humidityPercent(NAN),
      _valid(false) {}

void DhtSensor::init() {
    _dht.begin();
    _temperatureC = NAN;
    _humidityPercent = NAN;
    _valid = false;
}

bool DhtSensor::read() {
    float humidity = _dht.readHumidity();
    float temperature = _dht.readTemperature();

    if (isnan(humidity) || isnan(temperature)) {
        _valid = false;
        return false;
    }

    _humidityPercent = humidity;
    _temperatureC = temperature;
    _valid = true;
    return true;
}

float DhtSensor::getTemperatureC() const {
    return _temperatureC;
}

float DhtSensor::getHumidityPercent() const {
    return _humidityPercent;
}

bool DhtSensor::isValid() const {
    return _valid;
}
```

Listing 7.15 – Analog setpoint input header

```
/**
 * @file AnalogSetpointInput.h
 * @brief Potentiometer-based setpoint input wrapper.
 *
 * Encapsulates ADC reads and maps the raw value to a configurable
 * engineering range, such as a temperature setpoint in degrees Celsius.
 */

#ifndef ANALOG_SETPOINT_INPUT_H
#define ANALOG_SETPOINT_INPUT_H

#include <Arduino.h>

class AnalogSetpointInput {
public:
    /**
     * @brief Construct a setpoint input.
     * @param adcPin Analog pin connected to the potentiometer wiper.
     * @param minValue Engineering value at ADC 0.
     * @param maxValue Engineering value at ADC max.
     * @param adcResolution ADC resolution in bits, 10 for Arduino Mega.
     */
    AnalogSetpointInput(uint8_t adcPin, float minValue, float maxValue,
                        uint8_t adcResolution = 10);

    /** @brief Configure the ADC pin. */
    void init();

    /** @brief Read raw ADC value. */
    uint16_t readRaw();

    /** @brief Read and map the current value to the configured range. */
    float readValue();

    /** @brief Update the mapped engineering range. */
    void setRange(float minValue, float maxValue);

    /** @brief Return the last raw ADC sample. */
    uint16_t getLastRaw() const;

    /** @brief Return the last mapped setpoint value. */
    float getLastValue() const;

private:
    uint8_t _adcPin;
    float _minValue;
    float _maxValue;
};
```

```

    uint16_t _adcMax;
    uint16_t _lastRaw;
    float _lastValue;
};

#endif // ANALOG_SETPOINT_INPUT_H

```

Listing 7.16 – Analog setpoint input implementation

```

/**
 * @file AnalogSetpointInput.cpp
 * @brief Potentiometer-based setpoint input wrapper implementation.
 */

#include "AnalogSetpointInput.h"

AnalogSetpointInput::AnalogSetpointInput(uint8_t adcPin, float minValue,
                                          float maxValue, uint8_t adcResolution)
    : _adcPin(adcPin),
      _minValue(minValue),
      _maxValue(maxValue),
      _adcMax((1U << adcResolution) - 1U),
      _lastRaw(0),
      _lastValue(minValue) {}

void AnalogSetpointInput::init() {
    pinMode(_adcPin, INPUT);
    _lastRaw = 0;
    _lastValue = _minValue;
}

uint16_t AnalogSetpointInput::readRaw() {
    _lastRaw = analogRead(_adcPin);
    return _lastRaw;
}

float AnalogSetpointInput::readValue() {
    uint16_t raw = readRaw();
    float ratio = (float)raw / (float)_adcMax;
    _lastValue = _minValue + ratio * (_maxValue - _minValue);
    return _lastValue;
}

void AnalogSetpointInput::setRange(float minValue, float maxValue) {
    _minValue = minValue;
    _maxValue = maxValue;
}

uint16_t AnalogSetpointInput::getLastRaw() const {

```

```

    return _lastRaw;
}

float AnalogSetpointInput::getLastValue() const {
    return _lastValue;
}

```

Listing 7.17 – ON-OFF hysteresis controller header

```

/**
 * @file OnOffHysteresisController.h
 * @brief Heating-style ON-OFF controller with hysteresis.
 *
 * The controller turns the actuator ON when the measured value falls below
 * the lower threshold and turns it OFF when the measured value rises above
 * the upper threshold. Values inside the hysteresis band preserve the last
 * output, preventing relay chatter around the setpoint.
 */

#ifndef ON_OFF_HYSTERESIS_CONTROLLER_H
#define ON_OFF_HYSTERESIS_CONTROLLER_H

#include <Arduino.h>

enum HysteresisOutputState {
    HYSTERESIS_OUTPUT_OFF = 0,
    HYSTERESIS_OUTPUT_ON = 1
};

class OnOffHysteresisController {
public:
    /**
     * @brief Construct the controller.
     * @param setpoint Desired measured value.
     * @param hysteresisBand Full hysteresis band around the setpoint.
     */
    OnOffHysteresisController(float setpoint, float hysteresisBand);

    /** @brief Reset output to OFF. */
    void init();

    /** @brief Update controller configuration at runtime. */
    void setConfig(float setpoint, float hysteresisBand);

    /**
     * @brief Process one measured sample.
     * @return true when actuator command is ON.
     */
    bool update(float measuredValue);

```



```

    /** @brief Force the output to a known state. */
    void forceOutput(bool outputOn);

    /** @brief Current actuator command. */
    bool isOutputOn() const;

    /** @brief Current output state enum. */
    HysteresisOutputState getState() const;

    /** @brief Lower threshold where a heating actuator turns ON. */
    float getLowerThreshold() const;

    /** @brief Upper threshold where a heating actuator turns OFF. */
    float getUpperThreshold() const;

    /** @brief Configured setpoint. */
    float getSetpoint() const;

    /** @brief Configured full hysteresis band. */
    float getHysteresisBand() const;

private:
    float _setpoint;
    float _hysteresisBand;
    HysteresisOutputState _state;
};

#endif // ON_OFF_HYSTERESIS_CONTROLLER_H

```

Listing 7.18 – ON-OFF hysteresis controller implementation

```

/**
 * @file OnOffHysteresisController.cpp
 * @brief Heating-style ON-OFF controller with hysteresis implementation.
 */

#include "OnOffHysteresisController.h"

OnOffHysteresisController::OnOffHysteresisController(float setpoint,
                                                    float hysteresisBand)
    : _setpoint(setpoint),
      _hysteresisBand(hysteresisBand),
      _state(HYSTERESIS_OUTPUT_OFF) {}

void OnOffHysteresisController::init() {
    _state = HYSTERESIS_OUTPUT_OFF;
}

```

```

void OnOffHysteresisController::setConfig(float setpoint, float hysteresisBand) {
    _setpoint = setpoint;
    _hysteresisBand = hysteresisBand;
}

bool OnOffHysteresisController::update(float measuredValue) {
    if (_state == HYSTERESIS_OUTPUT_OFF &&
        measuredValue <= getLowerThreshold()) {
        _state = HYSTERESIS_OUTPUT_ON;
    } else if (_state == HYSTERESIS_OUTPUT_ON &&
        measuredValue >= getUpperThreshold()) {
        _state = HYSTERESIS_OUTPUT_OFF;
    }

    return isOutputOn();
}

void OnOffHysteresisController::forceOutput(bool outputOn) {
    _state = outputOn ? HYSTERESIS_OUTPUT_ON : HYSTERESIS_OUTPUT_OFF;
}

bool OnOffHysteresisController::isOutputOn() const {
    return _state == HYSTERESIS_OUTPUT_ON;
}

HysteresisOutputState OnOffHysteresisController::getState() const {
    return _state;
}

float OnOffHysteresisController::getLowerThreshold() const {
    return _setpoint - (_hysteresisBand * 0.5f);
}

float OnOffHysteresisController::getUpperThreshold() const {
    return _setpoint + (_hysteresisBand * 0.5f);
}

float OnOffHysteresisController::getSetpoint() const {
    return _setpoint;
}

float OnOffHysteresisController::getHysteresisBand() const {
    return _hysteresisBand;
}

```